

# SIH 2026: COOPERATIVE GIG SERVICES PLATFORM

Full Technical Architecture: User Lifecycles, Pricing & Welfare Engine, AI Allocation & Failure Recovery

**System Scope:** This blueprint provides the complete, authoritative operational logic implemented in the backend codebase. It details the journey from **Registration & Auth** to **Dynamic Booking Creation**, how **Commissions vs. Cooperative Welfare** are mathematically settled, how workers autonomously configure **Emergency & On-Demand Rates** above fair wage floors, the exact **Financial API Invoicing Formulation**, and how **AI Geo-Spatial Allocation and Automatic Re-allocation on Cancellation/Decline** guarantee zero service disruption.

## 1. User Lifecycle Workflow: Registration, Auth & Booking Creation

The platform provides dual streamlined access pathways tailored for Customers and Tradesmen, backed by JWT and Redis-cached active sessions:

| Phase                     | Customer Journey                                                                                              | Worker (Tradesman) Journey                                                                                           | Underlying Backend API & DB Action                                                             |
|---------------------------|---------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| Step 1: Registration      | Signs up via Name, Email, Phone, Password. Default saved address set with GPS Point [Lng, Lat].               | Registers with Trade Category (e.g. Plumbing), Experience, Hourly Base Rate, and Primary Labour Cooperative Society. | POST /api/auth/register<br>Creates User doc with role-based profile & default wallet balances. |
| Step 2: Login & Auth      | Inputs Email/Password or Phone OTP. Receives signed JWT access token.                                         | Inputs Credentials + Device ID. Verification status checked (isVerified: true required for job dispatch).            | POST /api/auth/login<br>Attaches JWT Bearer token + stores user:active-device:{id} in Redis.   |
| Step 3: Discovery         | Browses localized catalog in Hindi/English, or speaks to 'Hey Flexi' AI Voice Agent to describe issue.        | Toggles Availability (isOnline: true), broadcasts live GPS coordinates via mobile app.                               | GET /api/home/data?lang=hi<br>PUT /api/workers/me/availability                                 |
| Step 4: Booking Placement | Selects Service, chooses TimeSlot (Scheduled vs Emergency SOS), reviews transparent upfront invoice estimate. | Incoming booking alert received via Socket.io with audio chime & distance overlay.                                   | POST /api/bookings<br>Creates Booking in PENDING status with anti-ghost validation.            |

## 2. Platform Commission Model & Worker Autonomous Emergency Pricing

**Zero Exploitation Guarantee:** Unlike private aggregators taking 25-30% platform margin, this platform operates on cooperative principles: the platform fee is customizable (0% to admin margin) and 5% is diverted directly into the worker's own Welfare and Social Security Fund.

### Worker Autonomous Pricing for On-Demand & Emergency Services:

Workers are empowered to establish their own pricing structure while respecting Federation Fair Wage Floors:

- Category Rate Autonomy:** Workers can set custom rates per skill trade (workerProfile.categoryRates: [{ category: 'Plumbing', rate: 450 }]). When a customer books that specific worker, the custom rate supersedes the catalog base price.
- Statutory Federation Minimum Wage Floor:** A worker cannot set a rate below the Federation floor (e.g. Electrical: ₹400, Plumbing: ₹350), preventing destructive price-wars.
- Emergency SOS Surcharge Execution:** When a booking is marked isEmergency: true or bookingType: 'EMERGENCY\_SOS', the system automatically adds an emergency surcharge (Fixed ₹50 or 20% surcharge, whichever is higher). Workers receive this entire premium in addition to their standard fee as hazard/priority compensation.

## 3. API Workflow: Calculation, Charge Detection & Invoicing Engine

The financial engine separates the customer-facing invoice from the backend worker settlement cleanly through the following mathematical pipeline:

1. Customer Gross Payable Equation:

TotalAmount = BaseServiceFee + ExtraPartsTotal + CustomerPlatformFee + UrgentEmergencyFee

Where:

• BaseServiceFee = Selected Worker's Rate OR Service Catalog Default (Minimum Wage Floor enforced).

• CustomerPlatformFee = Admin-configured platform charge (can be 0 for 100% cooperative model).

• UrgentEmergencyFee = Surcharge applied only when isEmergency == true.

2. Worker Settlement & Cooperative Welfare Micro-Deduction Equation:

NetWorkerPayout = TotalAmount - (PlatformCommissionCut + CooperativeWelfareCut)

Where:

• PlatformCommissionCut = TotalAmount \* (workerCommissionPercent / 100) [Default: 0-5%]

• CooperativeWelfareCut = TotalAmount \* (cooperativeWelfarePercent / 100) [Strictly 5%]

• Welfare Destination: The 5% cut is atomically credited to WelfareAccount.balance and logged in WelfareTransaction. Once balance reaches 500, it automatically activates the worker's Accidental Insurance & Skilling Policy.

## 4. AI-Driven Workforce Allocation & Predictive Dispatch Engine

Task allocation is governed by a multi-variable algorithmic scoring model implemented in services/eligibleWorkers.js and controllers/aiController.js:

| Evaluation Factor             | Scoring Weight & Algorithmic Rule                                                                                                                      | Operational Impact                                                                          |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| Geo-Spatial Proximity         | Haversine formula calculates distance between job Point and worker GPS coordinates. Strict filter: distance <= worker.serviceRadiusKm (Default 10 km). | Ensures sub-15 minute arrival times, minimizing worker transit fuel expenditure.            |
| Trade & Skill Matching        | Multi-tier matching: Primary category -> categories[] -> skills[]. Verified certifications add a +15 score bonus.                                      | Eliminates misallocated jobs (e.g. electricians dispatched to plumbing bursts).             |
| Historical Performance        | Normalized formula: Score = (Rating / 5)*40 + Availability(25) + JobCompletionRate(20) + VerifiedBonus(15).                                            | Surfaces highest-trust cooperative members while giving new workers a fair baseline chance. |
| Predictive Cluster Allocation | Aggregates 24-hour booking velocity to identify municipal demand surges (e.g. Sector 18 plumbing surge), advising pre-standby positioning.             | Reduces customer peak wait times by over 40%.                                               |

## 5. Booking Cancellation, Failover & Atomic Re-allocation Engine

To ensure seamless service continuity without customer re-booking friction, the platform handles worker declines and customer cancellations with stateful event triggers:

### Scenario A: Worker Declines or Fails to Respond (Timeout):

1. Worker hits POST /api/bookings/:bookingId/decline with reason code (e.g. TOO\_FAR, UNAVAILABLE).

2. The system resets the booking status back to SEARCHING and disassociates the previous worker (booking.worker = null).

3. Immediate Automatic Cascade: The backend re-executes findEligibleWorkerIds() excluding the declining worker's ID, and immediately emits emergency:booking\_requested or booking:new\_available via Socket.io to the next nearest available eligible workers.

4. Preserved Booking Identity: The customer keeps the exact same Booking ID (e.g. #BK-260907-00827D129) and invoice, and is seamlessly notified via push: 'Assigning next nearest verified specialist...'

### Scenario B: Customer Initiates Cancellation (PATCH /api/bookings/:bookingId/cancel):

1. Pre-Arrival Check: If canceled before worker arrival, booking moves to CANCELLED with 0 cancellation penalty.

2. Instant Worker Availability Release: Live tracking Redis key (tracking:booking:{id}) is instantly deleted, and the worker's status is broadcasted as isAvailable: true so they immediately appear in incoming job pools.

3. Dual Socket & Push Alerts: Real-time event booking\_status\_update: CANCELLED emitted to the worker app, preventing wasted travel.

### Scenario C: Mid-Conversation Abort / Anti-Fake Protection:

If a customer stops responding during an AI voice session, Redis 60-second session TTL expires automatically (SESSION\_EXPIRED). Zero ghost bookings are ever written to MongoDB, protecting cooperative workers from fake dispatches.

Confidential - Official Technical Architecture Reference for Team & Mentors

Page 2 of 2